Experiment 10: Map Coloring using Constraint Satisfaction Problem (CSP)

Aim

To implement map coloring as a Constraint Satisfaction Problem (CSP), assigning colors to regions so that

adjacent regions have different colors.

Algorithm

1. Start

2. Define the regions and adjacency constraints.

3. Define the available colors.

4. Select an unassigned region.

5. Assign a color that does not conflict with already-colored neighbours.

6. If the assignment violates a constraint, backtrack.

7. Continue until all regions are assigned.

8. Display the valid coloring.

9. Stop.

Flowchart

Start

↓

Initialize

↓

Process / Search

↓

Goal / Condition Reached?

n n

Yes No

↓ ↓

Print Result Repeat

↓

Stop

Objective

• Understand constraint satisfaction problems.

• Apply backtracking to map coloring.

• Ensure neighboring regions receive different colors.

Program

```
def map_coloring(regions, colors, neighbours):
assignment = {}
def valid(region, color):
return all(assignment.get(n) != color for n in neighbours[region])
def backtrack():
if len(assignment) == len(regions):
return True
region = next(r for r in regions if r not in assignment)
for color in colors:
if valid(region, color):
assignment[region] = color
if backtrack():
return True
del assignment[region]
return False
if backtrack():
return assignment
return None
regions = ['A', 'B', 'C', 'D']
colors = ['Red', 'Green', 'Blue']
neighbours = {
'A': ['B', 'C'],
'B': ['A', 'C', 'D'],
'C': ['A', 'B', 'D'],
'D': ['B', 'C']
}
solution = map_coloring(regions, colors, neighbours)
```

```
print("Map Coloring Solution:")
for region in regions:
print(region, ":", solution[region])
```

Output

Map Coloring Solution:

A : Red

B : Green

C : Blue

D : Red

Result

The program was executed successfully and produced the expected result.

Conclusion

The experiment successfully demonstrated the corresponding Artificial Intelligence technique and its application